

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5

7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I, Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Judy Allen J

1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

Aim

To identify the functional dependencies and determine the candidate key for the relation STUDENT_COURSE.

Given

Relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

Where:

- SID = Student ID
- SName = Student Name
- CID = Course ID
- CName = Course Name
- Instructor = Course Instructor
- Grade = Grade obtained by the student

Functional Dependencies

1. $SID \rightarrow SName$
2. $CID \rightarrow CName$
3. $CID \rightarrow Instructor$
4. $(SID, CID) \rightarrow Grade$

Therefore,

$F = \{SID \rightarrow SName, CID \rightarrow CName, CID \rightarrow Instructor, SID, CID \rightarrow Grade\}$

Procedure / Analysis

Step 1: Determine the closure of {SID, CID}

Start with:

$(SID, CID)^+ = \{SID, CID\}$

Using:

$SID \rightarrow SName$

we get:

$(SID, CID)^+ = \{SID, CID, SName\}$

Using:

$CID \rightarrow CName, Instructor$

we get:

$(SID, CID)^+ = \{SID, CID, SName, CName, Instructor\}$

Using:

$SID, CID \rightarrow Grade$

we get:

$(SID, CID)^+ = \{SID, CID, SName, CName, Instructor, Grade\}$

The closure contains all attributes of the relation.

Therefore:

(SID, CID) is a superkey.

Since neither SID nor CID alone can determine all attributes, the combination is minimal.

Candidate Key Verification

Attribute Set	Closure	Candidate Key?
SID	$\{SID, SName\}$	No
CID	$\{CID, CName, Instructor\}$	No
SID, CID	$\{SID, CID, SName, CName, Instructor, Grade\}$	Yes

Result

Candidate Key = $\{SID, CID\}$

The identified functional dependencies are:

- $SID \rightarrow SName$
- $CID \rightarrow CName$
- $CID \rightarrow Instructor$
- $SID, CID \rightarrow Grade$

Conclusion

The relation $STUDENT_COURSE$ has a composite candidate key (SID, CID) . Student details depend on SID , while course details depend on CID . This dependency structure can cause redundancy and can be addressed through normalization.

2. Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Aim

To demonstrate the conversion of an unnormalized relation into First Normal Form (1NF) by eliminating repeating groups and ensuring atomic values.

Given

Consider the following unnormalized relation:

STUDENT(SID, SName, Courses)

Example:

SID	SName	Courses
S101	Arun	(DBMS, 85), (Python, 90)
S102	Priya	(DBMS, 88), (Java, 92)

The Courses attribute contains multiple values for a single student.

Procedure / Analysis

Step 1: Identify the Repeating Group

The Courses attribute contains multiple course-grade combinations.

For example:

S101 → (DBMS,85), (Python,90)

Therefore, Courses is a multi-valued attribute and is not atomic.

Step 2: Separate the Repeating Values

Create separate rows for each course taken by a student.

The relation becomes:

STUDENT_COURSE(SID, SName, Course, Grade)

SID	SName	Course	Grade
S101	Arun	DBMS	85
S101	Arun	Python	90
S102	Priya	DBMS	88
S102	Priya	Java	92

Step 3: Verify 1NF

The resulting relation satisfies 1NF because:

1. Every attribute contains atomic values.
2. There are no repeating groups.
3. Each row represents one student-course record.
4. Each column contains values of a single type.

Primary Key

The combination:

(SID, Course)

uniquely identifies each record.

Result

The relation is converted from UNF to:

STUDENT_COURSE(SID, SName, Course, Grade)

with:

Primary Key = (SID, Course)

Conclusion

The unnormalized relation is converted into 1NF by removing repeating groups and storing each value separately. All attributes now contain atomic values, satisfying the requirements of First Normal Form.

3. Given relation $R(A,B,C,D,E)$ with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.

Aim

To analyze the given relation for partial dependencies and determine whether decomposition is required to achieve Second Normal Form (2NF).

Given

Relation:

$R(A, B, C, D, E)$

Functional Dependencies:

1. $A \rightarrow BC$
2. $BC \rightarrow D$
3. $D \rightarrow E$

Procedure / Analysis

Step 1: Find the Candidate Key

Calculate the closure of A.

Start with:

$A^+ = \{A\}$

Using:

$A \rightarrow BC$

we get:

$A^+ = \{A, B, C\}$

Using:

$BC \rightarrow D$

we get:

$A^+ = \{A, B, C, D\}$

Using:

$D \rightarrow E$

we get:

$A^+ = \{A, B, C, D, E\}$

Therefore:

$A^+ = \{A, B, C, D, E\}$

Since A determines all attributes:

Candidate Key = A

Step 2: Check for Partial Dependency

A partial dependency occurs when a non-prime attribute depends on only a part of a composite candidate key.

Here, the candidate key contains only one attribute:

A

Therefore, partial dependency is not possible.

Step 3: Check 2NF

A relation is in 2NF if:

1. It is in 1NF.
2. No non-prime attribute is partially dependent on a candidate key.

Since A is a single-attribute key, there is no partial dependency.

The relation is therefore already in 2NF.

Observation

Although the relation satisfies 2NF, it contains further dependencies:

$BC \rightarrow D$

and

$D \rightarrow E$

These may require further decomposition when converting the relation to 3NF.

2NF Verification

Check	Observation
Candidate Key	A
Key Type	Single-attribute key
Partial Dependency	Not possible
2NF Status	Already in 2NF

Result

Candidate Key = A

2NF Relation = R(A, B, C, D, E)

No decomposition is required specifically for 2NF.

Conclusion

The given relation is already in 2NF because its candidate key A consists of a single attribute, eliminating the possibility of partial dependency. Further normalization may be required to remove transitive dependencies and achieve 3NF.

4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

Aim

To identify transitive dependencies and decompose the EMPLOYEE relation into Third Normal Form (3NF).

Given

Relation:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Functional Dependencies:

1. $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$
2. $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Original EMPLOYEE Relation

EmpID	EmpName	DeptID	DeptName	ManagerID
E101	Arun	D01	CSE	M01
E102	Priya	D01	CSE	M01
E103	Ravi	D02	Bioinformatics	M02

Procedure / Analysis

Step 1: Determine the Candidate Key

From:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$

and:

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

we obtain:

$\text{EmpID}^+ = \{\text{EmpID}, \text{EmpName}, \text{DeptID}, \text{DeptName}, \text{ManagerID}\}$

Therefore:

Candidate Key = EmpID

Step 2: Identify Transitive Dependency

The dependencies are:

$\text{EmpID} \rightarrow \text{DeptID}$

and:

DeptID $\rightarrow$ DeptName, ManagerID

Therefore:

EmpID $\rightarrow$ DeptID $\rightarrow$ DeptName, ManagerID

This represents a transitive dependency because DeptName and ManagerID depend on DeptID, which in turn depends on EmpID..

Step 3: Decompose the Relation

Create the following two relations.

EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID)

Primary Key:

EmpID

Foreign Key:

DeptID

DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID)

Primary Key:

DeptID

EMPLOYEE

EmpID	EmpName	DeptID
E101	Arun	D01
E102	Priya	D01
E103	Ravi	D02

DEPARTMENT

DeptID	DeptName	ManagerID
D01	CSE	M01
D02	Bioinformatics	M02

Step 4: Verify 3NF

In EMPLOYEE:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$

EmpID is the candidate key.

In DEPARTMENT:

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

DeptID is the candidate key.

Thus, the transitive dependency has been removed.

Result

The 3NF decomposition is:

EMPLOYEE(EmpID, EmpName, DeptID)

DEPARTMENT(DeptID, DeptName, ManagerID)

Conclusion

The original EMPLOYEE relation contains a transitive dependency through DeptID. Decomposition into EMPLOYEE and DEPARTMENT removes the transitive dependency, reduces redundancy, and produces relations satisfying 3NF.

5. Verify whether $R(A,B,C,D)$ with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.

Aim

To verify whether the given relation satisfies Boyce-Codd Normal Form (BCNF) and decompose it into BCNF if required.

Given

Relation:

$R(A, B, C, D)$

Functional Dependencies:

1. $AB \rightarrow C$
2. $C \rightarrow D$
3. $D \rightarrow A$

Procedure / Analysis

Step 1: Determine Candidate Keys

Closure of AB

$(AB)^+ = \{A, B\}$

Using $AB \rightarrow C$:

$(AB)^+ = \{A, B, C\}$

Using $C \rightarrow D$:

$(AB)^+ = \{A, B, C, D\}$

Therefore:

AB is a candidate key.

Closure of BC

$(BC)^+ = \{B, C\}$

Using $C \rightarrow D$:

$(BC)^+ = \{B, C, D\}$

Using $D \rightarrow A$:

$(BC)^+ = \{A, B, C, D\}$

Therefore:

BC is a candidate key.

Closure of BD

$$(BD)^+ = \{B, D\}$$

Using $D \rightarrow A$:

$$(BD)^+ = \{A, B, D\}$$

Using $AB \rightarrow C$:

$$(BD)^+ = \{A, B, C, D\}$$

Therefore:

BD is a candidate key.

Step 2: Check BCNF

A relation is in BCNF if for every non-trivial FD:

$$X \rightarrow Y$$

X must be a superkey.

$$AB \rightarrow C$$

AB is a candidate key.

Therefore, this dependency satisfies BCNF.

$$C \rightarrow D$$

C is not a superkey because:

$$C^+ = \{C, D, A\}$$

B is missing.

Therefore, $C \rightarrow D$ violates BCNF.

$$D \rightarrow A$$

D is not a superkey because:

$$D^+ = \{D, A\}$$

B and C are missing.

Therefore, $D \rightarrow A$ violates BCNF.

Hence, the original relation is not in BCNF.

Step 3: Decompose Using $C \rightarrow D$

Create:

$R1(C, D)$

and:

$R2(A, B, C)$

In $R1$, C is the key because:

$C \rightarrow D$

Therefore, $R1$ satisfies BCNF.

In $R2$, $AB \rightarrow C$ and AB is a candidate key. Therefore, $R2$ also satisfies BCNF.

BCNF Verification

Functional Dependency	Is Determinant a Superkey?	BCNF Status
$AB \rightarrow C$	Yes	✓ Satisfies
$C \rightarrow D$	No	✗ Violates
$D \rightarrow A$	No	✗ Violates

Result

The BCNF decomposition is:

$R1(C, D)$

$R2(A, B, C)$

Conclusion

The original relation is not in BCNF because C and D are determinants but are not superkeys. Decomposition into $R1(C,D)$ and $R2(A,B,C)$ removes the BCNF violations and produces BCNF relations.

6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

Aim

To experimentally verify whether a decomposition of a relation is lossless using the tabular Chase algorithm.

Given

Consider relation:

$R(A, B, C)$

Functional Dependency:

$A \rightarrow B$

Decomposition:

$R_1(A, B)$

$R_2(A, C)$

Procedure / Analysis

Step 1: Construct the Chase Table

Create one row for each decomposed relation.

Relation	A	B	C
R1	a	b	c2
R2	a	b2	c

The common attribute A receives the same distinguished symbol in both rows.

Step 2: Apply the Functional Dependency

Given:

$A \rightarrow B$

Both rows have the same value of A.

Therefore, their B values must become equal.

After applying the dependency:

Relation	A	B	C
R1	a	b	c2
R2	a	b	c

Since the Chase process produces a row containing the distinguished symbols a, b, c, the decomposition is lossless.

Step 3: Verify Lossless Join

A decomposition is lossless if the Chase process produces a row containing only distinguished symbols.

The second row contains:

a, b, c

Therefore, the decomposition is lossless.

Result

The decomposition:

$R(A,B,C) \rightarrow R1(A,B), R2(A,C)$

is verified as a lossless-join decomposition.

Conclusion

The Chase algorithm successfully verifies that the decomposition is lossless. The original relation can be reconstructed by joining the decomposed relations without losing information or introducing incorrect combinations.

7. Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

Aim

To identify the multi-valued dependencies in the TEACHER relation and decompose the relation into Fourth Normal Form (4NF).

Given

Relation:

TEACHER(TID, Subject, Hobby)

Assume that a teacher can have multiple subjects and multiple hobbies independently.

Example:

TID	Subject	Hobby
T1	DBMS	Reading
T1	DBMS	Music
T1	Python	Reading
T1	Python	Music

Procedure / Analysis

Step 1: Identify Multi-Valued Dependencies

For a teacher, subjects are independent of hobbies.

Original TEACHER Relation

TID	Subject	Hobby
T1	DBMS	Reading
T1	DBMS	Music
T1	Python	Reading
T1	Python	Music

Therefore:

$TID \twoheadrightarrow Subject$

and:

$TID \twoheadrightarrow Hobby$

These are multi-valued dependencies.

Step 2: Identify 4NF Violation

A relation is in 4NF if, for every non-trivial multivalued dependency:

$$X \twoheadrightarrow Y$$

X must be a superkey.

Here:

$$TID \twoheadrightarrow \text{Subject}$$

But TID alone does not identify a complete tuple because a teacher may have several subjects and several hobbies.

Therefore, the relation violates 4NF.

Step 3: Decompose the Relation

Separate the independent multi-valued attributes.

TEACHER_SUBJECT

TEACHER_SUBJECT(TID, Subject)

TID	Subject
T1	DBMS
T1	Python

TEACHER_HOBBY

TEACHER_HOBBY(TID, Hobby)

TID	Hobby
T1	Reading
T1	Music

The independent multi-valued attributes have been separated into two relations. This removes the 4NF violation and reduces unnecessary combinations.

Result

The 4NF decomposition is:

TEACHER_SUBJECT(TID, Subject)

TEACHER_HOBBY(TID, Hobby)

Conclusion

The original TEACHER relation contains independent multi-valued dependencies that cause redundancy. Decomposition separates subjects and hobbies into independent relations, thereby eliminating the 4NF violation.

8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

Aim

To demonstrate how join dependencies cause redundancy and decompose a relation into Fifth Normal Form (5NF), also called Project-Join Normal Form.

Given

Consider the relation:

SUPPLY(Supplier, Project, Part)

Assume that supplier-project, supplier-part, and project-part relationships are independent.

Example:

Supplier	Project	Part
S1	P1	Part1
S1	P1	Part2
S1	P2	Part1
S1	P2	Part2

Procedure / Analysis

Step 1: Identify Redundancy

The relation stores three types of information together:

1. Supplier works with Project.
2. Supplier supplies Part.
3. Project requires Part.

When these relationships are independent, storing all three together produces repeated combinations.

Step 2: Identify Join Dependency

The relation can be decomposed into:

SP(Supplier, Project)

Spart(Supplier, Part)

Ppart(Project, Part)

The original relation can be reconstructed using:

$SUPPLY = SP \bowtie Spart \bowtie Ppart$

Step 3: Create the Decomposed Relations

SP

SP(Supplier, Project)

Supplier	Project
S1	P1
S1	P2

SPart

SPart(Supplier, Part)

Supplier	Part
S1	Part1
S1	Part2

PPart

PPart(Project, Part)

Project	Part
P1	Part1
P1	Part2
P2	Part1
P2	Part2

Step 4: Verify Reconstruction

The original relation can be obtained through:

$SUPPLY = SP \bowtie SPart \bowtie PPart$

The decomposition separates independent relationships and avoids storing the same combinations repeatedly.

5NF Condition

A relation is in 5NF when every non-trivial join dependency is implied by the candidate keys of the relation.

Result

The 5NF decomposition is:

SP(Supplier, Project)

SPart(Supplier, Part)

PPart(Project, Part)

Conclusion

Join dependencies can cause redundancy when independent relationships are stored in a single relation. Decomposition into smaller relations removes this redundancy and helps achieve Fifth Normal Form (5NF).

9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Aim

To identify insertion, deletion, and update anomalies in a poorly designed college database and normalize the relation up to Third Normal Form (3NF).

Given

Consider the poorly designed relation:

COLLEGE(SID, SName, CID, CName, FacultyID, FacultyName, Grade)

Example:

SID	SName	CID	CName	FacultyID	FacultyName	Grade
S101	Arun	C01	DBMS	F01	Kumar	A
S102	Priya	C01	DBMS	F01	Kumar	B
S103	Ravi	C02	Python	F02	Meena	A

Update Anomaly

If the faculty name for F01 changes, multiple records must be updated.

Insertion Anomaly

A new course cannot be stored easily if no student has enrolled in it.

Deletion Anomaly

Deleting the only student enrolled in a course may also remove the only stored information about that course.

Functional Dependencies

1. $SID \rightarrow SName$
2. $CID \rightarrow CName, FacultyID$
3. $FacultyID \rightarrow FacultyName$
4. $SID, CID \rightarrow Grade$

Candidate Key:

(SID, CID)

Procedure / Analysis

Step 1: Identify Update Anomaly

If the faculty name associated with F01 changes, every record containing F01 must be updated.

If one record is not updated, inconsistent faculty information will occur.

Therefore, an update anomaly exists.

Step 2: Identify Insertion Anomaly

Suppose a new course C03 is introduced but no student has enrolled in it.

The current relation cannot store the course information without also inserting student-related information.

Therefore, an insertion anomaly exists.

Step 3: Identify Deletion Anomaly

Suppose the only student enrolled in C02 is deleted.

The deletion may also remove the only stored information about the course and its faculty.

Therefore, a deletion anomaly exists.

Step 4: Normalize to 3NF

Create the following relations.

STUDENT

STUDENT(SID, SName)

STUDENT

SID	SName
S101	Arun
S102	Priya
S103	Ravi

Primary Key:

SID

COURSE

COURSE(CID, CName, FacultyID)

COURSE

CID	CName	FacultyID
C01	DBMS	F01
C02	Python	F02

Primary Key:

CID

FACULTY

FACULTY(FacultyID, FacultyName)

FACULTY

FacultyID	FacultyName
F01	Kumar
F02	Meena

Primary Key:

FacultyID

ENROLLMENT

ENROLLMENT(SID, CID, Grade)

ENROLLMENT

SID	CID	Grade
S101	C01	A
S102	C01	B
S103	C02	A

Primary Key:

(SID, CID)

Foreign Keys:

- SID references STUDENT(SID)
- CID references COURSE(CID)

3NF Verification:

The decomposition removes the transitive dependency $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$ (or the corresponding course/faculty dependency in this college schema) and separates independent entities into individual relations. Each resulting relation has a key that determines the non-key attributes, and the relations satisfy 3NF.

Result

The normalized 3NF schema is:

STUDENT(SID, SName)

COURSE(CID, CName, FacultyID)

FACULTY(FacultyID, FacultyName)

ENROLLMENT(SID, CID, Grade)

Conclusion

The original college relation suffers from insertion, deletion, and update anomalies due to redundant information. Decomposition into STUDENT, COURSE, FACULTY, and ENROLLMENT relations removes redundancy and produces a well-structured 3NF database.

10. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Aim

To compute the closure of attribute sets using functional dependencies and determine the candidate key of a relation.

Given

Consider relation:

$R(A, B, C, D, E)$

Functional Dependencies:

1. $A \rightarrow B$
2. $B \rightarrow C$
3. $CD \rightarrow E$
4. $E \rightarrow A$

Procedure / Analysis

Step 1: Compute A^+

Start with:

$A^+ = \{A\}$

Using:

$A \rightarrow B$

we get:

$A^+ = \{A, B\}$

Using:

$B \rightarrow C$

we get:

$A^+ = \{A, B, C\}$

No dependency can determine D.

Therefore:

$A^+ = \{A, B, C\}$

Since A^+ does not contain all attributes, A is not a candidate key.

Step 2: Compute D^+

Start with:

$$D^+ = \{D\}$$

No functional dependency can be applied.

Therefore:

$$D^+ = \{D\}$$

D is not a candidate key.

Step 3: Compute $(CD)^+$

Start with:

$$(CD)^+ = \{C, D\}$$

Using:

$$CD \rightarrow E$$

we get:

$$(CD)^+ = \{C, D, E\}$$

Using:

$$E \rightarrow A$$

we get:

$$(CD)^+ = \{A, C, D, E\}$$

Using:

$$A \rightarrow B$$

we get:

$$(CD)^+ = \{A, B, C, D, E\}$$

Therefore:

$$(CD)^+ = \{A, B, C, D, E\}$$

Since the closure contains all attributes, CD is a superkey.

Neither C nor D alone determines all attributes.

Therefore:

CD is a candidate key.

Step 4: Compute E^+

Start with:

$$E^+ = \{E\}$$

Using:

$$E \rightarrow A$$

we get:

$$E^+ = \{E, A\}$$

Using:

$$A \rightarrow B$$

we get:

$$E^+ = \{E, A, B\}$$

Using:

$$B \rightarrow C$$

we get:

$$E^+ = \{E, A, B, C\}$$

D is missing.

Therefore:

$$E^+ = \{A, B, C, E\}$$

E is not a candidate key.

Closure Summary

Attribute Set	Closure	Candidate Key
A	$\{A, B, C\}$	No
D	$\{D\}$	No
CD	$\{A, B, C, D, E\}$	Yes
E	$\{A, B, C, E\}$	No

Since $(CD)^+ = \{A, B, C, D, E\}$, the attribute set CD determines all attributes of the relation. Since neither C nor D alone determines all attributes, $\{C, D\}$ is a minimal superkey and therefore a candidate key.

Result

The candidate key is:

$\{C, D\}$

because:

$(CD)^+ = \{A, B, C, D, E\}$

Conclusion

Attribute closure is used to determine all attributes functionally dependent on a given attribute set. The closure of CD contains every attribute of R, and CD is minimal. Therefore, $\{C,D\}$ is the candidate key of the given relation.